

CPU 与 OS 内核：软硬件协作边界

系统调用、地址翻译、抢占、DMA 与并发原子性的责任边界

408 专题手册 · 计组 × 操作系统桥梁篇

核心模型：硬件机制与内核策略通过接口协作

CPU/设备提供可被强制执行的局部机制，
OS 内核配置这些机制、维护全局状态，并基于目标选择策略。

交接过程：

硬件机制 $\xrightarrow{\text{被 OS 配置}}$ 受控事件与状态转换 $\xrightarrow{\text{内核处理/策略}}$ 新的系统状态

分析软硬件边界时依次追问：

1. 哪个**物理能力**必须由硬件保证？
2. OS 预先配置了哪些表、寄存器、权限或队列？
3. 什么**事件**触发软硬件交接？
4. 硬件至少保存/检查什么，才能安全把控制权交给内核？
5. 内核读取哪些状态、执行什么策略、修改哪些数据结构？
6. 返回前，谁负责恢复可继续执行的体系结构状态？

目录

1 第一原则：机制、策略与状态归属	2
1.1 把“硬件提供机制，OS 提供策略”展开成三层	2
1.2 六问法：跨计组与 OS 的统一分析模板	2
2 控制权与特权级：用户代码怎样高速运行而不失控	2
2.1 先建立架构无关的入口协议	2
2.2 System Call、Exception、Interrupt 的共同点与差异	3
2.3 x86 例子必须拆成两条路径：不要把 IDT 与 SYSCALL 混成一条	3
3 虚拟内存：MMU 快速执行翻译，OS 决定映射意味着什么	4
3.1 地址转换是一个“硬件执行、软件定义”的典型契约	4
3.2 Page Fault 的本质：硬件发现“当前翻译不能按正常路径完成”	4
3.3 TLB：不要再写“换 CR3 就一定全部清空”	5

3.4 Accessed/Dirty 位：思想要保留，实现不要绝对化	5
4 时间虚拟化与抢占：硬件制造调度机会，OS 决定是否换人	5
4.1 OS 如何保证死循环程序也必须交还控制权	5
4.2 调度不是 timer IRQ 的固定后半段	6
4.3 Context Switch 的软硬件边界	6
5 I/O 与 DMA：硬件搬运、驱动编排、调度隐藏等待	6
5.1 一次块 I/O 的正确分层	6
5.2 DMA 的真正含义：卸载“逐字搬运”，不是免费传输	7
5.3 DMA 地址不一定等于 CPU 物理地址	7
6 并发与同步：原子性、内存顺序与“睡眠”必须分层理解	7
6.1 硬件到底为锁提供了什么？	7
6.2 硬件原子指令为什么还不够？	8
6.3 Fast path / Slow path 再次出现	8
7 五个维度重新统一：同一个“配置 - 触发 - 接管 - 返回”结构	8
8 综合题：一次阻塞 read() 怎样横跨计组与 OS	9
8.1 第一阶段：控制权交接	9
8.2 第二阶段：资源引用与地址保护	9
8.3 第三阶段：快路径失败	10
8.4 第四阶段：设备独立工作	10
8.5 第五阶段：中断、唤醒与重新调度	10
9 高频误区：从入门表述回到准确边界	10
10 408 跨计组/OS 解题检查表	11
11 与相关专题的接口	12
工程边界资料	12

1 第一原则：机制、策略与状态归属

1.1 把“硬件提供机制，OS 提供策略”展开成三层

软硬件协作至少包含三层：

层次	回答的核心问题	典型代表与工程例子
Hardware Mechanism	机器能够且必须可靠地做什么？	特权级检查、MMU 地址翻译、原子 RMW 指令、异常/中断向量表、定时器计数、DMA 搬运
Kernel Mechanism	OS 如何把硬件能力封装成可复用系统机制？	Trap Entry 汇编、页表管理、上下文切换 (<code>switch_to</code>)、Sleep/Wakeup 队列、驱动框架、互斥锁
Policy	多个合法选择中应该选哪个？	进程调度算法 (CFS/MLFQ)、页面置换算法 (LRU/CLOCK)、I/O 调度策略、资源回收与降级策略

第三个必须补上的问题：状态到底归谁保存？

例如一次系统调用横跨三类状态：

- CPU 当前寄存器中的体系结构状态；
- 内核栈或 trap frame 中的被保存执行状态；
- PCB/TCB、页表、文件表、调度队列中的内核管理状态。

很多综合题真正考的不是术语，而是“此刻这份状态在哪里”。

1.2 六问法：跨计组与 OS 的统一分析模板

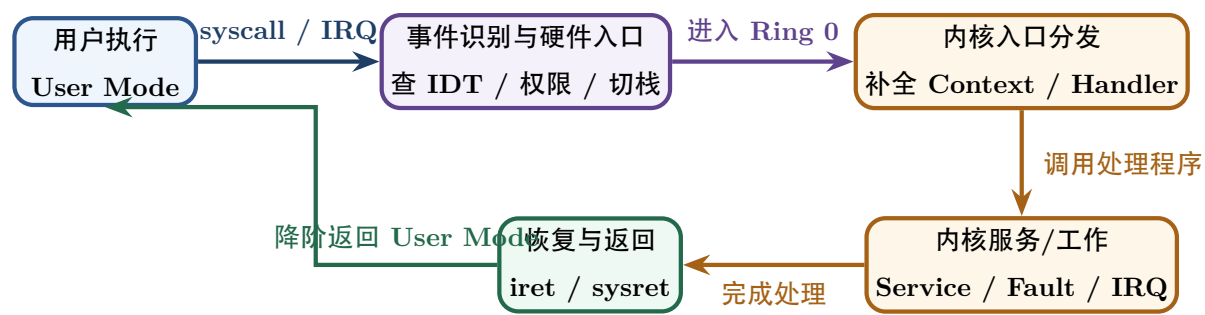
看到软硬件边界题，固定问六件事

1. 事件是什么？ syscall、page fault、timer interrupt、I/O completion、lock contention？
2. 硬件先做什么？ 权限检查、最小状态保存、地址转换、原子读改写、发中断？
3. OS 预先配置了什么？ 入口地址、页表、定时器、DMA 描述符、等待队列？
4. 内核接管后做什么？ 判断合法性、记账、调度、分配、唤醒、更新映射？
5. 什么时候可能发生 context switch？ 进入内核本身不等于切换实体。
6. 返回时什么状态必须恢复？ PC/SP/权限、页表上下文、寄存器、设备/内存可见性？

2 控制权与特权级：用户代码怎样高速运行而不失控

2.1 先建立架构无关的入口协议

无论具体 ISA 怎样实现，一次从用户执行进入内核处理，大体都可压成：



硬件必须至少保证：

- 用户不能任意伪造更高特权级；
- 进入内核后存在一个可信入口；
- 有足够状态使原执行流未来能够继续或被终止；
- 非法特权操作能够被捕获，而不是直接破坏全局机器状态。

OS 则负责：

- 启动阶段配置入口、权限、栈/上下文结构；
- 把低级入口统一整理成 trap frame / saved context；
- 判断进入原因并调用具体 handler；
- 决定返回原执行流、发送信号/终止，还是进入调度器。

2.2 System Call、Exception、Interrupt 的共同点与差异

入口类型	与当前指令关系	典型触发源头	内核接管后首先关心与处理
系统调用 (Syscall)	同步、主动发起	用户执行 <code>syscall</code> / <code>trap</code> / <code>int</code> 指令	校验调用号、提取参数、检查用户内存合法性、返回结果
异常/故障 (Exception)	同步、被动触发	除零、缺页 (<code>#PF</code>)、非法指令、保护异常	异常能否修复并重发指令（如缺页/COW）？还是终止进程？
外部中断 (Interrupt)	异步于当前程序	Timer 芯片、网卡、磁盘 DMA 控制器	哪个硬件设备完成请求？需要把谁从 Blocked 变 Ready？

408 核心：进入内核态不等于上下文切换

系统调用、异常或中断首先造成的是**控制权/特权级转换**。只有内核进一步发现当前执行实体不能继续、应该被抢占，或者主动选择另一个 runnable 实体时，才发生上下文切换。

2.3 x86 例子必须拆成两条路径：不要把 IDT 与 SYSCALL 混成一条

关键误区辨析：传统陷阱门 vs 快速 syscall

“CPU 查 IDT、自动切内核栈并压入 SS/ESP/EFLAGS/CS/EIP”适合描述**传统 x86 中断/陷阱门跨特权级的经典路径**，但 **x86-64 的快速 SYSCALL 指令并不等价于 IDT interrupt gate**。Linux x86-64 也存在多种不同入口，它们的调用约定与栈处理并不完全相同。

因此手册采用：

- **408/架构无关层**：硬件完成可信入口与必要状态转换，内核入口保存其余上下文；
- **x86 interrupt/trap gate 例子**：IDT、TSS/IST、硬件形成异常/中断入口 frame；
- **x86-64 SYSCALL 例子**：使用专用系统调用入口约定，栈与寄存器整理需要专门入口代码处理，不应套用“IDT 自动压五项”的模板。

工程边界

Linux x86 文档明确区分 64-bit system_call、INT80 compatibility、SYSENTER、普通 interrupt、APIC interrupt 与架构异常等多种入口；不同入口具有不同 calling convention。学习时应先掌握共同机制，再把具体寄存器布局当作架构实现细节。

3 虚拟内存：MMU 快速执行翻译，OS 决定映射意味着什么

3.1 地址转换是一个“硬件执行、软件定义”的典型契约

一次普通 load/store 可以抽象为：

$$VA \xrightarrow{\text{TLB / page walk}} PA \xrightarrow{\text{cache/memory}} data.$$

核心关注点	硬件 Hardware / MMU 职责	软件 OS Kernel 职责
页表数据结构	读取架构规定的页表根指针 (如 CR3) 与 PTE 格式	在物理内存中分配页表树，建立、修改与释放映射关系
地址翻译过程	TLB Lookup；未命中时执行硬件 Page Walk	决定 VPN 到 PFN 的逻辑映射，设置权限/脏位/存在位
访问合法性校验	执行 PTE 权限检查 (User/Supervisor, R/W)，违规报 Fault	根据 VMA 区域语义判断是否为 COW、按需分配或非法越界
翻译结果加速	TLB 自动缓存 $VA \rightarrow PA$ 翻译结果	当映射更新或释放物理页时，执行 TLB Flush / Shootdown
内存页面回收	——（硬件不参与策略选择）	维护全局内存压力，运行 CLOCK / LRU 页面置换与 Swap 写回

3.2 Page Fault 的本质：硬件发现“当前翻译不能按正常路径完成”

统一流程：

memory access → translation/protection failure → fault entry → kernel diagnoses cause

然后可能分叉：

- 合法按需页

COW write

swapped/not resident

非法访问

→ allocate/map → retry

→ copy/update PTE → retry

→ I/O + block → retry

→ signal/terminate

关键思想

MMU 不知道“这是不是 COW”“这个页该从哪个文件读取”“是否应该牺牲别的页”。它只知道**当前表项与当前访问是否满足体系结构规则**。语义解释与策略选择属于 OS。

3.3 TLB：不要再写“换 CR3 就一定全部清空”

408 经典模型可以先理解为：切换不同地址空间需要切换页表上下文，并处理旧 TLB translation，带来额外开销。

现代 x86 则存在 PCID 等机制，可以让不同地址空间的 TLB 项带有上下文标识，避免每次切换都简单粗暴地整表清空；具体失效规则取决于硬件功能与内核策略。

边界修正

“TLB hit 一定 1 cycle”“CR3 更新必然完全清空 TLB”“上下文切换会把 CPU Cache 清空”都不应作为普遍结论。408 应掌握**为什么翻译缓存能加速、为什么映射变化必须保持 TLB 一致性**，而不是背某代 CPU 的固定时延。

3.4 Accessed/Dirty 位：思想要保留，实现不要绝对化

在 x86 等体系结构中，硬件可在页表项中维护 accessed/dirty 类状态，OS 可利用这些信息近似判断热度和写回需求。但不同 ISA 对这些位的硬件支持方式并不完全一致。

因此跨架构的正确表达是：

硬件/软件共同维护“最近是否访问、是否修改”这类页状态信息

它们为页面回收和写回策略提供观测信号，而不是策略本身。

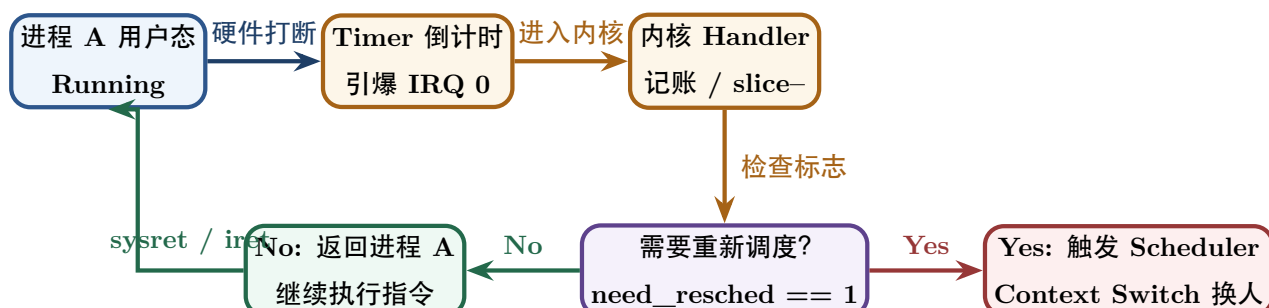
4 时间虚拟化与抢占：硬件制造调度机会，OS 决定是否换人

4.1 OS 如何保证死循环程序也必须交还控制权

用户程序：

```
while (1) { }
```

并发程序并不会主动 syscall。真正保证 OS 最终重新获得 CPU 的关键机制是**可编程定时器 + 外部中断 + 特权入口**。



硬件提供“我可以打断正常用户控制流并进入可信内核入口”；OS 才负责“这次是否应该换人、换给谁”。

4.2 调度不是 timer IRQ 的固定后半段

408 可以理解成：timer interrupt 提供抢占机会，调度器根据策略选择 runnable task。

更严格地说，具体内核可能先做计时/统计，只设置 reschedule 标记，并在合适的 scheduling point 才真正调用调度器。因此：

timer interrupt $\neq$ 必然 context switch

工程边界：Linux：固定 HZ 不是唯一时间模型

传统教材喜欢用“每 1ms/10ms 一个时钟中断”建立直觉；现代通用 OS 还广泛使用 local APIC timer、high-resolution timer 与 tickless/nohz 等机制减少无意义周期性 tick。408 核心仍然是：硬件时间事件创造可抢占点，内核维护调度状态与策略。

4.3 Context Switch 的软硬件边界

一次真正的切换至少需要：

1. 保存旧执行实体恢复所需的 CPU 状态；
2. 更新旧实体的内核管理状态；
3. 调度器选择新的 runnable entity；
4. 必要时切换地址空间上下文；
5. 恢复新实体的执行状态；
6. 返回其用户态或继续内核态执行。

不要把某个 Linux 汇编例程保存的“callee-saved registers 列表”背成 CPU 上下文的通用定义：有些寄存器可能已经在 trap frame 中保存，有些由 ABI/入口路径决定。

5 I/O 与 DMA：硬件搬运、驱动编排、调度隐藏等待

5.1 一次块 I/O 的正确分层

Process $\rightarrow$ Kernel/VFS/Block Layer $\rightarrow$ Driver $\rightarrow$ Device/DMA $\rightarrow$ Interrupt/Completion $\rightarrow$ Wakeup

阶段	Hardware / Device	Kernel / Driver
准备	暴露 MMIO/PIO 寄存器、DMA capability	建缓冲/descriptor, 做 DMA mapping, 写设备寄存器/队列
传输	device/DMA 发起内存总线事务	当前线程可阻塞, 调度其他 runnable 工作
完成	更新设备状态, 产生 interrupt 或 completion	ISR/driver 确认完成、结束 request、唤醒 waiter
继续	—	waiter 通常变 Ready/Runnable, 未来由 scheduler 再获得 CPU

5.2 DMA 的真正含义：卸载“逐字搬运”，不是免费传输

DMA 的核心收益是：CPU 不需要执行一条条 load/store 指令替设备搬完整数据块。

但 DMA 仍然：

- 占用 memory/interconnect 带宽；
- 可能经过 IOMMU 地址翻译；
- 必须处理设备与 CPU 对内存的可见性/一致性；
- 在非一致性平台上可能需要 cache synchronization；
- 需要 CPU/driver 设置 descriptor、处理完成和错误。

不要写“DMA 不占 CPU，所以 CPU 一定 100% 利用率”

更准确的说法是：**DMA 显著减少 CPU 对数据搬运本身的指令参与**，使 CPU 能执行其他工作；系统最终利用率仍取决于是否存在 runnable workload、内存带宽竞争、驱动开销和设备等待等。

5.3 DMA 地址不一定等于 CPU 物理地址

现代系统可能存在 IOMMU 或其他 DMA 地址转换，因此 driver 交给设备的 DMA address 不一定能被 CPU 直接当作 physical address 使用。

这再次体现：

OS 建立映射与权限；硬件执行受控地址访问

6 并发与同步：原子性、内存顺序与“睡眠”必须分层理解

6.1 硬件到底为锁提供了什么？

多核环境中，OS/运行时需要某种**原子 Read-Modify-Write** 能力建立竞争点，例如：

- compare-and-swap / cmpxchg；
- test-and-set / exchange；
- fetch-and-add；

- LR/SC 一类保留-条件存储机制。

硬件还需要给出明确的 memory ordering / fence 语义，使多个核心对共享内存的观察能够按照同步算法要求建立顺序约束。

三个概念不要揉成一个

- **Cache coherence**: 多个 cache 对同一 cache line 的副本怎样保持一致；
- **Atomicity**: 某次 RMW 是否表现为不可被另一核心拆开的单一竞争操作；
- **Memory ordering**: 不同 load/store 在其他核心看来允许以什么顺序可见。

“有 MESI”不等于“程序天然顺序一致”，也不等于“锁算法自动正确”。

6.2 硬件原子指令为什么还不够？

假设 CAS 只能告诉线程：

lock free? → acquire success/fail.

如果等待时间很长，失败线程一直 CAS：

Spin

会浪费 CPU。

因此 OS/运行时进一步提供：

Atomic primitive + wait queue + sleep/wakeup + scheduler

形成 blocking mutex、semaphore、condition variable 等高级同步原语。

6.3 Fast path / Slow path 再次出现

- 无竞争：用户态原子操作直接成功，快路径；
- 有竞争且等待很短：可能 spin；
- 有竞争且需要阻塞：进入内核，加入等待队列并 schedule，慢路径。

Linux futex 正是理解这种分层的优秀工程例子：常见无竞争路径依赖用户空间原子状态，真正需要等待/唤醒时才通过 futex syscall 进入内核。

7 五个维度重新统一：同一个“配置 - 触发 - 接管 - 返回”结构

核心维度的应用场 景	硬件 (Hardware) 提供 / 执行的物 理机制	软件 (OS Kernel) 配置 / 维护 / 决策的 逻辑	触发界面的 关键事件
特权与系统调用	硬件 Privilege Checks；可信入口 向量；自动切换内核栈并保护基础 断点	启动初始化入口向量与权限；在内核栈补 全保存 TrapFrame；系统调用分发与返 回/调度	syscall / trap

核心维度的应用场景	硬件 (Hardware) 提供 / 执行的物理机制	软件 (OS Kernel) 配置 / 维护 / 决策的逻辑	触发界面的关键事件
虚拟内存管理	MMU 自动硬件 Page Walk; TLB 映射缓存; PTE 权限校验; 产生缺页异常	分配物理页表树; 定义 VMA/对象映射语义; Page Fault Handler; 页面回收与 TLB 刷新	#PF / TLB miss
分时抢占机制	硬件 Timer 倒计时计数; 向 CPU 引爆外部硬件中断 (IRQ)	时间片记账与更新; 维护 Ready 队列; 触发 need_resched 并执行 Scheduler 切换	timer IRQ
存储与外设 I/O	暴露 MMIO/PIO 接口; DMA 控制器独立总线数据传输; 中断引爆	驱动程序装载; 配置 DMA 映射 buffer; 请求队列编排; 线程 Blocked 与唤醒 Wakeup	I/O completion
并发与同步原语	硬件原子 RMW 指令 (CMPXCHG/LL-SC); Memory Fence 屏障; Cache 一致性协议	构建 Mutex/Semaphore 高级原语; 维护等待队列; 阻塞睡眠 sleep() 与唤醒 wake_up()	contention / signal

真正统一的结构

五个主题表面差异很大，但底层都是：

OS 先配置规则/状态 → 硬件高速执行 common case → 特殊事件触发边界 → 内核接管慢路径 → 修复/决策/更新状态 → 重新回到高速执行

这就是理解“CPU × OS 协作”的最强统一视角。

8 综合题：一次阻塞 read() 怎样横跨计组与 OS

假设用户线程调用：

```
read(fd, buf, 4096);
```

且目标数据不在页缓存，需要真实设备 I/O。

8.1 第一阶段：控制权交接

用户代码准备参数并执行系统调用入口。
硬件/体系结构负责：建立受控入口的最低保证。
内核入口负责：补全当前执行流上下文、读取 syscall number/arguments、进入 read 逻辑。

8.2 第二阶段：资源引用与地址保护

内核根据：
 $fd \rightarrow open\ file\ description \rightarrow file/inode$

定位打开实例，并验证用户 buffer 是否可以合法写入。

这里已经同时使用了：

- 进程资源表；
- VFS 抽象；
- 虚拟地址空间与保护；
- 用户态/内核态边界。

8.3 第三阶段：快路径失败

如果页缓存 hit，数据可能很快返回。

现在假设 miss：

read → block layer/driver → DMA request.

当前线程等待设备：

Running → *Blocked*.

内核调度另一个 runnable 实体。

8.4 第四阶段：设备独立工作

设备/DMA 在内存与设备之间传输数据；CPU 可以执行其他任务，但 DMA 会使用内存系统与片上/系统互连资源。

完成后设备产生 interrupt/completion。

8.5 第五阶段：中断、唤醒与重新调度

CPU 进入 interrupt handler；driver 完成 request bookkeeping，并唤醒等待者：

Blocked → *Ready*.

注意：

wakeup ≠ immediately running

只有之后 scheduler 选中它：

Ready → *Running*.

线程才真正继续 read 的内核路径，最终返回用户态。

一道 read() 已经把两门课串起来

这条路径同时考察：系统调用入口、CPU 特权级、中断、PCB/线程状态、调度、虚拟内存保护、VFS、I/O、DMA、Cache/页缓存与阻塞/唤醒。以后遇到跨计组 + OS 综合题，都应该训练成这种“时间线上谁接管、状态在哪里、谁负责什么”的推演。

9 高频误区：从入门表述回到准确边界

容易背错的习惯说法	更准确稳健的物理与工程模型
“syscall 时 CPU 一定查 IDT、自动压五个寄存器并切内核栈”	这是把某类 x86 interrupt/trap gate 的经典路径泛化了；快速 syscall 指令拥有不同入口约定。408 掌握“可信入口 + 保存恢复上下文”即可。
“进入内核态就是进程切换”	模式切换只是当前执行流进入内核；只有 scheduler 真正换了执行实体才是 context switch。
“换 CR3 就完全清空 TLB”	经典模型强调切换地址空间会带来翻译缓存开销；现代硬件可用 PCID/ASID 类机制保留不同上下文的 translation。
“上下文切换会清空 CPU Cache”	Cache 内容不会因为“进程名变了”自动整体清空；性能损失更多来自 working set 改变、cache/TLB 命中率下降与其他微结构效应。
“TLB hit 就是 1 cycle”	命中很快，但具体 latency 是微架构参数；408 应掌握层次与额外访存次数。
“A/D 位都由 MMU 自动维护”	x86 等架构常有硬件维护；跨架构应理解为页访问/脏状态由软硬件共同建立和使用。
“CPU 每个固定 1ms/10ms 必然被 IRQ0 打断”	经典周期 timer 是教学模型；现代系统还有 APIC timer、high-resolution timer 与 tickless 机制。
“DMA 不占任何系统资源”	DMA 减少 CPU 搬运指令，但仍消耗 memory/interconnect 带宽，并涉及 mapping、coherency、completion。
“I/O 完成中断后等待进程立刻运行”	handler 通常只是使 waiter Ready/Runnable；什么时候 Running 由 scheduler 决定。
“有原子指令就有高质量 mutex”	原子 RMW 只提供竞争原语；阻塞 mutex 还需要等待队列、sleep/wakeup、调度与公平策略。
“MESI 保证并发程序顺序正确”	coherence、atomicity、memory ordering 是不同层次；同步算法还需要正确的原子操作与 fence/order 语义。

10 408 跨计组/OS 解题检查表

软硬件分工十问

1. 当前执行在 user 还是 kernel?

2. 当前 CPU 上是谁？某个进程/线程，还是 interrupt context?

3. 是同步事件还是异步事件?

4. 硬件自动检查/保存/转换了什么?

5. OS 在事件发生前预先配置了什么?

6. 内核 handler 要读取哪些寄存器/表项/设备状态?

7. 当前实体还能继续吗？若不能，是 Ready 还是 Blocked?

8. 是否真的发生 context switch / page switch / I/O wait?

9. 这条路径是快路径还是 slow/exception path?

10. 最终恢复时，哪些体系结构状态和内核不变量必须成立?

四类最常见综合题

- **系统调用/中断 + 调度**: 何时进入内核? 是否必然切换?
- **TLB/Page Table + Page Fault**: 硬件翻译到哪一步, OS 从哪一步接管?
- **DMA + Interrupt + Process State**: 谁搬数据? 谁阻塞? 谁唤醒?
- **Atomic Instruction + Semaphore/Mutex**: 硬件保证哪一步原子, OS 为什么还要等待队列和调度?

11 与相关专题的接口

责任边界

- 进程与调度专题定义执行实体、状态转换和调度; 这里仅说明硬件事件怎样把控制权交给内核。
- 虚拟内存专题定义页表、TLB、缺页异常和页面生命周期; 这里只保留 MMU 与内核的交接。
- 并发专题定义锁、信号量与等待条件; 这里只说明原子指令、内存顺序和睡眠机制的分层。
- I/O 专题定义请求、DMA、完成与唤醒; 这里只说明设备、CPU 与驱动的责任边界。

工程边界资料

以下资料用于核对 Linux/x86 的实现边界, 不作为 408 必背 API:

- Linux x86 Kernel Entries: 不同 syscall/interrupt/exception 入口拥有不同 calling convention。
[kernel.org: x86 Kernel Entries](https://kernel.org/doc/Documentation/x86/entries.txt)
- Linux x86 PTI/PCID: PCID 可降低页表切换时简单全量 TLB flush 的成本。
[kernel.org: x86 PTI](https://kernel.org/doc/Documentation/x86/pti.txt)
- Linux DMA API: DMA address、coherent/non-coherent mappings 与同步要求。
[kernel.org: DMA API](https://kernel.org/doc/Documentation/dma-api.txt)
- Linux DMA mapping guide: 设备与 CPU 的缓存可见性、DMA buffer 语义。
[kernel.org: DMA mapping guide](https://kernel.org/doc/Documentation/dma-mapping-guide.txt)
- Linux futex manual: 用户态原子快路径与内核 wait/wake 慢路径。
[man7.org: futex\(2\)](https://man7.org/linux/man-pages/futex(2).html)

核心结论

CPU/设备负责把规则强制执行得足够快、足够可靠;
OS 内核负责决定规则怎样配置、状态怎样解释、资源怎样分配、失败以后怎样继续。

系统能力来自双方稳定而清晰的契约, 不是任何一方单独完成全部工作。